JAVA INTERVIEW PREPARATION

CHAPTER 67

TREES AND
BINARY SEARCH TREES

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Trees and Binary Search Trees: Structure, Invariants, and Ordered Data

Senior / Lead Java Developer Reading Chapter

Trees model hierarchy by giving each node zero or more children and, except for the root, exactly one parent in the logical structure. They appear in syntax trees, user-interface components, file systems, organization models, routing tables, indexes, and ordered collections. Interview questions use binary trees because two child references keep the code compact, but the senior-level skill is broader: define the structural invariant, choose the traversal that matches the required order, and make recursion depth and mutation behavior explicit.

A binary search tree adds an ordering invariant. That invariant enables search, insertion, deletion, range queries, and ordered iteration without scanning every element. It also creates failure modes: duplicate policy, inconsistent comparators, skewed shape, and mutation of ordering keys can all make a logically plausible implementation incorrect.

Tree vocabulary describes relationships

```
                8                root, depth 0
              /     \
             3       10          internal nodes
            / \        \
           1   6        14          leaves include 1, 6, 14

subtree rooted at 3: {3, 1, 6}
path from 8 to 6: 8 -> 3 -> 6
```

Depth is normally the number of edges from the root to a node. Height is the number of edges on the longest downward path from a node to a leaf, although some APIs count nodes instead. An empty-tree height may be defined as -1 or 0. Neither convention is universally correct; an interview answer should declare the convention and apply it consistently.

The degree of a node is its number of children. A leaf has no children. A subtree consists of a node and all descendants reachable below it. A tree with `n` nodes has `n - 1` parent-child edges. If arbitrary references point back to ancestors, the physical object graph may contain cycles even when the logical domain is called a tree; traversal then requires a visited set or a stricter ownership model.

Binary tree is not binary search tree

A binary tree merely limits each node to at most two children. A binary search tree, or BST, defines how keys are ordered. A common strict policy is:

```
for every node N:
all keys in N.left  < N.key
all keys in N.right > N.key
```

The rule applies to entire subtrees, not only immediate children. If duplicates are allowed, the implementation needs a stable policy: duplicates always go left, always go right, or one node stores a count/list of equal values. A balanced ordered-map implementation often replaces an existing value when comparison returns zero.

This is why the following tree is invalid even though each parent appears locally plausible:

```
          10
         /  \
        5    15
            /  \
           6    20

6 is below the right subtree of 10, so it must be greater than 10.
```

Traversal order is an execution contract

Depth-first traversals differ by when the current node is processed:

- preorder: node, left, right;
- inorder: left, node, right;
- postorder: left, right, node.

For a BST, inorder traversal produces keys in comparator order. Preorder is useful when the parent must be handled before descendants, such as copying structure or emitting an opening representation. Postorder is useful when children must be completed before the parent, such as computing subtree aggregates or deleting a tree.

```
static void inorder(Node node, List<Integer> output) {
    if (node == null) return;
    inorder(node.left, output);
    output.add(node.key);
    inorder(node.right, output);
}
```

The recursive version mirrors the definition and is often the clearest. Its auxiliary space is O(h), where h is tree height, because each active call represents an ancestor. On a balanced tree, h is O(log n); on a skewed tree it is O(n), which can overflow the Java stack.

An iterative inorder traversal makes that state explicit:

```
static List<Integer> inorder(Node root) {
    List<Integer> result = new ArrayList<>();
    Deque<Node> ancestors = new ArrayDeque<>();
    Node current = root;

    while (current != null || !ancestors.isEmpty()) {
        while (current != null) {
            ancestors.push(current);
            current = current.left;
        }
        current = ancestors.pop();
        result.add(current.key);
        current = current.right;
    }
    return result;
}
```

Level-order traversal uses a queue and visits nodes by depth. Capturing the queue size at the start of a level preserves the frontier boundary. This is useful for minimum-depth problems, per-level output, and width calculations.

Search and insertion follow one path

BST search compares the desired key with the current node and chooses one subtree. It does not need to inspect the other subtree because the ordering invariant proves the key cannot be there.

```
static Node find(Node root, int key) {
    Node current = root;
    while (current != null) {
        if (key == current.key) return current;
        current = key < current.key ? current.left : current.right;
    }
    return null;
}
```

Search and insertion take O(h), not automatically O(log n). The logarithmic claim depends on height being logarithmic. Inserting sorted input into an unbalanced BST produces a linked-list shape.

```
balanced enough:            sorted insertion into plain BST:

       4                    1
     /   \                   \
    2     6                   2
   / \   / \                   \
  1  3  5  7                   3 -> 4 -> 5

height O(log n)             height O(n)
```

Insertion must also define equality. For a set, an equal key may leave the tree unchanged. For a map, comparison equality commonly updates the associated value. For a multiset, a count can be incremented. Hiding that decision inside generic pointer code produces subtle data-loss or duplicate-order bugs.

Deletion has three structural cases

Deleting a BST node is difficult because ordering and reachability must both survive.

1. A leaf can simply be removed. 2. A node with one child can be replaced by that child. 3. A node with two children is replaced by its inorder successor or predecessor, then that replacement is removed from its original position.

The successor is the smallest node in the right subtree, so it has no left child. That property reduces the second removal to one of the simpler cases.

```
static Node delete(Node root, int key) {
    if (root == null) return null;

    if (key < root.key) {
        root.left = delete(root.left, key);
    } else if (key > root.key) {
        root.right = delete(root.right, key);
    } else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        Node successor = minimum(root.right);
        root.key = successor.key;
        root.value = successor.value;
        root.right = delete(root.right, successor.key);
    }
    return root;
}

static Node minimum(Node node) {
    while (node.left != null) node = node.left;
    return node;
}
```

Copying a successor key is safe only when the node's identity is not externally meaningful and all associated payload is copied consistently. If other objects retain node references, transplanting nodes may be required instead. Parent pointers, subtree sizes, colors, or balance factors must also be updated. Production tree

implementations centralize such rewiring because partially updated metadata silently corrupts later operations.

Validating a BST requires inherited bounds

Checking only `node.left.key < node.key < node.right.key` misses violations deeper in a subtree. The recursive call must inherit every ancestor constraint.

```
static boolean validBst(Node node, Long lower, Long upper) {
    if (node == null) return true;

    long key = node.key;
    if (lower != null && key <= lower) return false;
    if (upper != null && key >= upper) return false;

    return validBst(node.left, lower, key)
            && validBst(node.right, key, upper);
}
```

Using nullable `Long` bounds avoids overflow at `Integer.MIN_VALUE` and `Integer.MAX_VALUE`. Another valid method performs inorder traversal and ensures strict increase, but it must maintain previous state correctly and reflect the duplicate policy. Bounds generally communicate the global invariant more directly.

For generic keys, bounds use the same `Comparator` as insertion. Validation with natural ordering while insertion used a custom comparator verifies the wrong tree.

Balanced trees protect the height guarantee

A self-balancing BST performs local rotations after updates so height remains logarithmic. Rotations change shape while preserving inorder order.

```
right rotation around 30:

       30                    20
      /                     /  \
     20          ->        10   30
    /
   10

inorder before and after: 10, 20, 30
```

AVL trees maintain a strict height-balance condition, typically giving very fast lookup at the cost of more update rebalancing. Red-black trees maintain color and path constraints that allow somewhat looser balance with efficient updates. Java's `TreeMap` and `TreeSet` are red-black-tree-based ordered collections. A senior candidate normally needs to explain the guarantees and tradeoffs, not reproduce all rotation cases from memory unless the role specifically demands it.

Balanced trees provide O(log n) search, insertion, and deletion, plus ordered navigation such as floor, ceiling, predecessor, successor, and range views. Hash tables normally provide faster average exact lookup, but do not inherently provide sorted iteration or efficient order-based queries.

Range queries are where ordered maps become especially valuable. A query for every key from `fromInclusive` to `toExclusive` first locates the lower boundary in O(log n), then walks only the `k` matching entries, for O(log n + k) work. Java's `NavigableMap` exposes operations such as `lowerEntry`, `floorEntry`, `ceilingEntry`, `higherEntry`, and bounded views. These views are backed by the original map: mutation through the view changes the map, and keys outside the range are rejected. Treating a view as an independent snapshot is a common API-level mistake.

Comparator semantics are part of correctness

`TreeMap` decides key identity through ordering: if the comparator returns zero, the keys occupy the same map position even if `equals` says they differ. The comparator should therefore usually be consistent with equality.

```
Comparator<Customer> byLastNameOnly =
        Comparator.comparing(Customer::lastName);

// Two different customers named Singh compare as zero.
```

Using that comparator for a set would collapse distinct customers. A safer ordering adds stable tie-breakers that match the intended identity.

```
Comparator<Customer> order = Comparator
        .comparing(Customer::lastName)
        .thenComparing(Customer::firstName)
        .thenComparing(Customer::id);
```

Comparators should be transitive and antisymmetric in sign. Avoid subtraction such as `a.priority - b.priority`, which can overflow; use `Integer.compare`. Ordering keys must not mutate while stored in an ordered collection. A key whose comparison result changes may become unreachable because the tree will search along a path different from the one used at insertion.

Business ordering and business identity are not always the same. If people are displayed by name but uniquely identified by ID, use one comparator for presentation and an ID-based key for storage rather than forcing one relation to serve both purposes.

Null handling must also be deliberate. Natural ordering cannot compare a null key, and modern `TreeMap` usage should not rely on historical implementation quirks. A comparator can explicitly place nulls first or last, but allowing null may weaken the domain model and complicate range boundaries. Prefer rejecting missing business keys unless null has a defined ordering meaning.

Subtree computations: height, size, and diameter

Many tree problems use postorder because a parent's answer depends on child results.

```
static int heightInNodes(Node node) {
    if (node == null) return 0;
    return 1 + Math.max(heightInNodes(node.left),
                        heightInNodes(node.right));
}
```

The diameter is the longest path between any two nodes. At each node, a path through that node combines left height and right height. The global maximum may lie entirely inside a child subtree, so the traversal must return height while updating or returning diameter as a second result.

```
record Metrics(int height, int diameter) {}

static Metrics metrics(Node node) {
    if (node == null) return new Metrics(0, 0);

    Metrics left = metrics(node.left);
    Metrics right = metrics(node.right);
    int height = 1 + Math.max(left.height(), right.height());
    int throughNode = left.height() + right.height();
    int diameter = Math.max(throughNode,
            Math.max(left.diameter(), right.diameter()));
    return new Metrics(height, diameter);
}
```

This single traversal is O(n). Recomputing height separately at every node can degrade to O(n squared) on a skewed tree. The general optimization is to return all information the parent needs from one postorder visit.

Order statistics and augmented trees

The kth smallest key in a BST can be found by inorder traversal in O(h + k) time without augmentation. If kth, rank, or select operations are frequent, each node can store subtree size.

```
node.size = 1 + size(node.left) + size(node.right)
```

If the left subtree contains `L` nodes, the current node has rank `L + 1` within its subtree. Comparing `k` with that rank selects left, current, or right in O(h). Insertions, deletions, and rotations must update sizes; augmentation improves queries but increases the mutation invariant.

The same pattern supports interval trees, where nodes store the maximum endpoint in a subtree, and other search structures. The senior design lesson is that cached metadata trades cheaper queries for more complex writes and more ways to become inconsistent.

Lowest common ancestor depends on structure

For a general binary tree, the lowest common ancestor can be found by recursively searching both subtrees. If one target is found on each side, the current node is the answer. If both are on one side, that subtree returns the answer upward. The contract must clarify whether both targets are guaranteed to exist; otherwise the result needs existence tracking.

For a BST, ordering offers a simpler path. If both keys are smaller, go left. If both are larger, go right. Otherwise the current node is the split point and therefore the lowest common ancestor. This is O(h) and demonstrates the value of using the strongest available invariant rather than applying a generic tree algorithm.

Building and serializing trees

Traversal data reconstructs a tree only under specific assumptions. Preorder plus inorder can reconstruct a binary tree when keys are unique. Inorder alone cannot determine shape. Preorder plus postorder is generally ambiguous unless additional constraints, such as a full binary tree, are supplied.

A robust serialization includes null markers so structure is unambiguous.

```
tree:          2
              / \
             1   3

preorder with nulls: 2, 1, #, #, 3, #, #
```

Production formats should also define escaping, schema version, duplicate rules, numeric range, maximum depth, and malformed-input behavior. Recursive deserialization of attacker-controlled depth can exhaust the stack. Limits should be validated before or during construction.

Immutable trees and safe publication

Immutable nodes make a tree easier to share across threads because readers never observe partial mutation after safe publication. Updating an immutable BST creates new nodes along the root-to-update path while sharing untouched subtrees.

```
old root --------------------> unchanged right subtree
   \
    old left subtree

new root -- new path nodes --> updated leaf
   \
    shares unchanged branches with old version
```

For a balanced tree, an update can copy O(log n) nodes and preserve the old snapshot for concurrent readers. This is structural sharing, not a full deep copy. It suits configuration snapshots, compiler structures, and versioned state. The tradeoffs are more allocation and the need for an immutable balancing strategy.

A mutable tree shared across threads requires synchronization around compound operations and rotations. Making child references `volatile` is insufficient: a rotation updates several related links and metadata that must appear atomically as one valid structure. Mature concurrent ordered indexes use carefully designed algorithms rather than ad hoc volatile pointers.

Snapshot semantics should be named explicitly. A reader of a persistent immutable root sees one coherent version even while a writer publishes a newer root. A reader of a mutable tree protected only per operation may observe different versions across two calls. If a workflow requires a stable range scan, use a lock, versioned snapshot, or collection with documented snapshot behavior rather than assuming an iterator freezes the structure. Standard fail-fast iterators are debugging aids and do not provide a concurrency guarantee.

Trees in production systems

In-memory BSTs are not the same as database indexes. Storage engines often use B-trees or B+ trees with high branching factors so each page access yields many keys and tree height stays small. Cache-aware and disk-aware layouts matter more than the binary shape familiar from interviews.

Java applications encounter trees in dependency graphs, ASTs, JSON models, routing rules, and ordered maps. Before choosing a tree, ask whether the workload needs hierarchy, sorted navigation, prefixes, spatial ranges, or merely exact lookup. A hash map may be better for equality lookup; a trie may be better for prefixes; a heap may be better when only the next minimum matters.

Observability should reflect structural health. For a custom mutable index, useful measurements include node count, height, balance distribution, update rate, range-query selectivity, allocation, and lock contention. A correctness check can sample inorder ordering and metadata consistency, but a full validation on every request may be too expensive.

Testing tree algorithms

Tree tests need shapes, not only values:

- empty tree and one node;
- completely left- and right-skewed trees;
- balanced and nearly balanced trees;
- minimum and maximum integer keys;
- duplicates according to the declared policy;
- deletion of root, leaf, one-child, and two-child nodes;
- deep trees that expose recursion limits;
- comparators with equivalent values and tie-breakers.

Property-based testing is especially effective. After any sequence of BST updates, inorder output should be ordered and should equal the reference multiset or map. Stored subtree sizes should match recursively counted sizes. Every reachable child should have exactly one expected parent when parent pointers are used. For balanced trees, balance and color invariants should hold after every operation.

Common senior-level traps

- Assuming every binary tree is a BST.
- Validating only immediate children instead of inherited bounds.
- Claiming O(log n) for a plain BST without a balance guarantee.
- Ignoring duplicate-key policy.
- Recomputing subtree height at every node and accidentally producing O(n squared).
- Using recursion without considering adversarial depth.
- Copying only the successor key during deletion and forgetting its payload or metadata.
- Using a comparator inconsistent with intended identity.
- Mutating fields that participate in ordering while a key is stored.
- Assuming two traversals always uniquely reconstruct a tree.
- Treating volatile child pointers as an atomic tree update.

A strong interview answer

Start by identifying whether the structure is a general tree, binary tree, or ordered BST, and state the duplicate and height assumptions. Choose traversal from dependency order: preorder for parent-first work, postorder for child-derived results, inorder for BST order, and level order for depth frontiers. State complexity in terms of height before simplifying it. For updates, explain the pointer and metadata invariants. For Java ordered collections, include comparator consistency and mutable-key risks. Finally, discuss recursion depth, concurrency, and whether a different structure better matches the workload.

Chapter summary

Trees encode hierarchy, while BSTs additionally encode order. Traversals are different execution orders over the same structure. Search and mutation cost depend on height, which balancing controls. Global validation requires inherited bounds, and deletion must preserve reachability plus all augmented metadata. Comparators define key identity in Java's ordered collections. Immutable structural sharing can make snapshots safer, while mutable concurrent trees require coordinated multi-link updates.

Revision Notes

- A binary tree limits children; a BST also imposes a subtree-wide ordering rule.
- Declare height convention and duplicate policy.
- Preorder is parent-first; inorder is ordered for a BST; postorder is children-first.
- Recursive auxiliary space is O(h), and a skewed tree can overflow the stack.
- BST search, insertion, and deletion are O(h), not inherently O(log n).
- Two-child deletion uses a successor or predecessor and must preserve payload and metadata.
- Validate a BST with inherited bounds or a correctly implemented inorder check.
- Rotations change shape while preserving inorder order.
- Red-black and AVL trees keep height O(log n) with different balance/update tradeoffs.
- `TreeMap` identity is determined by comparison returning zero.
- Comparators should be transitive, safe from arithmetic overflow, and normally consistent with equality.
- Postorder can return multiple subtree metrics in one O(n) traversal.
- Augmented metadata speeds queries but expands the update invariant.
- Serialization requires enough null/shape information and depth limits.
- Immutable trees can update through path copying and share untouched subtrees.

Likely interview question: Why is checking each node against its children insufficient to validate a BST?

Because a descendant is constrained by every ancestor. A node in the right subtree of 10 must remain greater than 10 even if it is the left child of 15. Passing lower and upper bounds through recursion preserves all inherited constraints.

Likely interview question: When is BST lookup O(n)?

When tree height is O(n), as in a plain BST that receives already sorted keys and becomes skewed. The correct general complexity is O(h). A balancing scheme such as red-black or AVL maintains O(log n) height.

Likely interview question: Why can TreeSet lose an apparently distinct object?

If its comparator returns zero for two objects, the set treats them as the same ordered key even when `equals` returns false. The comparator must include tie-breakers that represent the intended identity, or storage should use a different key.

Likely interview question: Recursive or iterative traversal?

Use recursion when the tree depth is safely bounded and clarity is the priority. Use an explicit stack when depth may be large, when processing must pause and resume, or when stack-overflow risk matters. Both use O(h) traversal state for depth-first traversal.